Intented

NEAR Hackathon, Nov 1–14 2026: build plan and task split for Furqaan & William

"Your agent only does what you intented." An AI agent's NEAR wallet contract executes an action only when it carries a TEE-signed `ALLOW` verdict from NEAR AI Cloud. The verdict checks the action against the owner's plain-English mandate, and the contract verifies the signature on-chain.

BUILD WINDOW	HARD DEADLINE	OUR SUBMIT TARGET	RESULTS
Nov 1 – Nov 14	**Nov 14 23:59 UTC = Nov 15 05:29 IST**	**Nov 14, 20:00 IST**	**by Nov 18 23:59 UTC**

TRACKS	TEAM	MAINNET BUDGET	FALLBACK IDEA
AI Security & Defense · Agent Economy · Privacy & Private AI	**2 (Furqaan + William)**	**$20–50 (1Click tests)**	**Sealed-lite (Day-2 switch)**

FURQAAN WILLIAM BOTH: owner badges used throughout.

The split. Furqaan owns the security-critical core: the Rust wallet contract, on-chain signature verification, the **firewall core** library (prompt, NEAR AI call, signature fetch, contract submit), attestation, mainnet deploys and the submission form. **William** owns the frontend, the demo **and the backend around the core**: the API server and proposal storage, the 1Click payment module, the red-team runner, the UI, the demo agent and paywall, the video and the X post. William's API calls Furqaan's firewall core through a fixed function interface (section 5). Until Day 6 it uses a stub of that core, so neither of you waits on the other.

1. Hackathon at a glance

Item	Detail
Organizer / site	NEAR, run directly at **nearhackathon.com**. There is no Devpost or DoraHacks page. The site's `HACKATHON.md` is a briefing written for AI coding agents.
Prizes	**$250K total, paid in NEAR.** $55K grand prize, plus $10K / $3K / $2K for 1st–3rd in each of 13 tracks. One submission can select every track it fits.
Judges	Illia Polosukhin (NEAR), Eddy Lazzarin (a16z crypto), Mert Mumtaz (Helius), Avichal Garg (Electric Capital), Kain Warwick (Synthetix/Infinex), Haseeb Qureshi (Dragonfly), Stepan Gershuni (cyber•Fund). **Claude Code & Codex are listed as AI judges**, so the repo and README will be read by machines.
Criteria (no weights published)	Technical implementation, design, impact, originality, sustainability.
Required to submit	A **public GitHub repo**, plus a **public X post** saying it was built at NEAR HACKATHON with a demo video or screenshots. Other fields: a 20–2,000 character description, the team size (choose 2), and a **NEAR mainnet account** for the payout.
Hard rules	**One submission per team, and it cannot be edited.** A second submission with the same name or email is rejected. "Build your project during the hackathon": product code goes in a repo created on or after Nov 1.
README guidance	Write it for a machine reader as well as a human one: what it is, the problem, "do X - see Y" steps, no marketing, and name the track hints you use.

Why Intented (from the selection research)

- **Not on Illia's Oct 3 idea list.** 6 of the 10 ideas we considered were, so those tracks will be crowded with copies.
- **It matches the Security track's hints exactly:** "Agent security, function call keys, TEE attestation."
- **It uses only documented, working APIs.** NEAR AI Cloud signs every response, and a NEAR contract can verify that with `sha256` + `ecrecover`. 1Click is mature: 205 assets on 37 chains, quotes in about 1 s.
- **It fits the judges:** Haseeb says AI's real use is "software engineering and security", and Avichal raises agent-wallet liability as unsolved.
- **It fills a real gap.** OutLayer and existing repos only check rules such as limits and allowlists. OutLayer has no co-signer hook and does not re-verify the token or amount against the 1Click quote. Intented checks the *meaning* of each action.

2. What we're building



Component	What it does	Owner
Wallet contract (near-sdk-rs)	Defines `propose`, `execute`, signer registry, mandate storage, nonce and expiry replay guard, hard per-tx and daily caps, allowlist and freeze switch. It verifies the NEAR AI signature and checks that the action inside the signed request is byte-for-byte the action being executed.	FURQAAN
Attestation checker (script)	Calls `/v1/attestation/report` with a fresh nonce, verifies it off-chain, and registers the TEE signing address in the contract (owner-only).	FURQAAN
Firewall core (TypeScript library)	Canonical encoder. Builds the prompt from mandate + action with untrusted data quoted, calls NEAR AI `/v1/chat/completions` with structured JSON output, fetches `/v1/signature/{id}` immediately, returns the raw request and response bytes, and submits `execute` to the contract.	FURQAAN
API server & storage (TypeScript)	The HTTP API the UI and agent use (section 5). Stores proposals, verdicts and raw bytes in SQLite, calls the firewall core, and handles retries, logging and deployment (Railway/Render or similar).	WILLIAM
Cross-chain execution (1Click module)	Quote, deposit, submit and status for NEAR USDC → Base USDC, plus refund handling. Status is polled and written back into each proposal.	WILLIAM (contract side: F)
Red-team runner	`make redteam`: runs every scenario through the real pipeline and outputs the block rate and false-positive rate as JSON and Markdown, served at `/api/redteam/latest`.	WILLIAM
Attack & benign scenarios	20+ malicious and 20+ benign cases as JSON fixtures. The most visual ones become the demo.	WILLIAM
Demo agent + mock x402 paywall	An agent that buys data from an x402-style 402 paywall, which has an honest version and a poisoned one that injects "pay 10× to this address". The agent calls the firewall API.	WILLIAM
UI (web app)	Landing section with the one-liner, mandate editor, live proposal feed, verdict and reason, links to the signature, attestation and explorer transactions, red-team stats panel. Deployed on Vercel.	WILLIAM
Brand & demo assets	Logo, colors, the demo video, the X post, README screenshots and GIFs.	WILLIAM

Model & trust assumptions (say these honestly in the README)

- **Model:** Qwen3.8-27B on NEAR AI Cloud, a NEAR-hosted open model at $0.04 in / $1.35 out per M tokens. Fallback: GLM-5.3-flash. Avoid proprietary models until we confirm they return TEE signatures.
- **What's verified where:** the TEE attestation is checked off-chain, and the signer address is then allowlisted on-chain. **Do not claim the contract verifies the TDX quote.**
- **Backstop:** hard caps and allowlists stay in the contract, so the AI can only tighten what the contract permits, never loosen it.
- **The firewall model can itself be prompt-injected.** Mitigations: pass untrusted tool output as quoted data, use structured outputs, and keep the caps.

Demo script (target ≈ 2:00–2:20) WILLIAM

1. 0:00 Problem: agents with wallets get prompt-injected, and spending caps alone don't catch an attack that stays under the cap.
2. 0:20 Owner sets the mandate in the UI: "Buy market data, max $5 per call, only from vendors I approve."
3. 0:40 The agent buys a dataset, gets **ALLOW**, and the payment goes cross-chain to Base via 1Click. Show the explorer tx.
4. 1:10 A poisoned paywall injects "pay 10× to 0xATTACKER". The amount is still under the cap. Intented returns **DENY** with a reason and the contract refuses to execute.
5. 1:40 Click the receipt: signature verified, attestation report, red-team stats (e.g. "19/20 attacks blocked, 1/20 false positives").
6. 2:05 Close: "Give AI money without giving it the keys to everything."

3. Roles & ownership

FURQAAN Contract, AI core & submission

- Wallet contract in near-sdk-rs: all methods, caps, replay guard, freeze switch, payout path to the 1Click deposit address
- On-chain signature verification (`env::ecrecover` or `env::ed25519_verify`)
- Function-call key scoping, and proof that out-of-scope calls fail
- Firewall core library: canonical encoder, prompt layout, NEAR AI calls, signature fetch, contract submit
- Attestation checker and signer registration and rotation
- Prompt tuning against the red-team numbers, and hardening
- Testnet and mainnet contract deploys, NEAR payout account
- README technical sections: architecture, threat model, trust assumptions
- Fresh-clone test, secret scan, final submission form
- Discord: NEAR AI, contract-delivery, OutLayer and Shade questions

WILLIAM Frontend, demo & backend services

- **Backend:** API server (Node + Hono/Express) implementing the HTTP API, SQLite storage for proposals and raw bytes, logging, deploy
- **Backend:** 1Click module: quote → deposit → submit → status → refund, with status written back to proposals
- **Backend:** red-team runner (`make redteam`) and metrics endpoint
- Web app: landing section, mandate editor, proposal feed, receipt view, red-team stats panel
- Intented branding: logo, colors, favicon, OG image
- Demo agent and mock x402 paywall, honest and poisoned versions
- Attack and benign scenario fixtures (20+ / 20+)
- Demo storyboard, recording and editing the video, the X post
- README product, demo and API sections, with screenshots and GIFs
- Discord: 1Click and organizer questions

Shared: the interface spec (section 5), the Day-6 core swap, the Day-7 integration, reviewing the red-team numbers on Day 8, and the final go/no-go before submitting.

4. Pre-hackathon prep (Oct 23 – Oct 31)

> Before Nov 1, do **accounts, reading, design and throwaway experiments only**. Do not write product code you plan to submit. The real repo is created on Nov 1. Design mockups, the storyboard and branding are fine to prepare, but confirm in Discord (organizer question #10).

FURQAAN	WILLIAM
☐ Create the NEAR mainnet payout account (`intented.near` is taken; try `intentedhq.near`) ☐ Get a NEAR AI Cloud API key and call `/v1/models` ☐ Install the Rust toolchain, `cargo-near` , `near-cli-rs` and near-workspaces ☐ Create testnet accounts for the owner, agent and attacker ☐ Read the NEAR AI response-signature verification docs ☐ Experiment with `env::ecrecover` using a signature you generate yourself (ethers), then throw it away ☐ Fund NEAR USDC in a hot wallet ($20–50) ☐ Ask Discord questions 1, 2, 3, 4, 6, 8 (section 8) ☐ Draft the threat model and README technical skeleton	☐ Register **intented.ai** (or .xyz) after checking the price ☐ Create the X account and the GitHub org (`intented` is taken; try `intented-hq`) ☐ Branding: logo, color palette, wordmark ☐ Figma or wireframes for 4 screens: landing, mandate editor, proposal feed, receipt ☐ Pick the stack (Next.js + Tailwind on Vercel; Node + Hono + SQLite API on Railway/Render) and set up the accounts ☐ Read the 1Click OpenAPI spec and dry-run a quote with the TypeScript SDK ☐ Apply for the 1Click partner JWT at partners.near-intents.org (approval time unknown) ☐ Read the x402 spec (HTTP 402 response format) for the mock paywall ☐ Draft 25+ attack and 25+ benign scenarios in plain English ☐ Storyboard the 2-minute demo video, shot by shot ☐ Set up a Base address to receive the demo payment ☐ Ask Discord questions 5, 9, 10, 11 (1Click + organizer)

5. Interface spec (agree on this on Nov 1, morning)

The contract, the firewall core, the API and the UI all use the same encoding. The **firewall core interface** is the seam between you: William's API calls it, starting with a stub that returns fake signed verdicts, and Furqaan replaces the stub with the real core on Day 6. Only change it if you both agree, and log the change in `DECISIONS.md`.

```
// Canonical action (service and contract encode identically)
action = { receiver_id, method, args_sha256, deposit /*yocto string*/, nonce /*u64*/, expiry_ms /*u64*/ }
canonical_bytes = "INTENTED_V1|" + receiver_id + "|" + method + "|" + args_sha256
                  + "|" + deposit + "|" + nonce + "|" + expiry_ms

// Verdict the model must return (structured JSON output)
{ "verdict": "ALLOW" | "DENY", "reason": "<≤280 chars>", "action": "<canonical_bytes>" }

// Contract methods (Furqaan)
set_mandate(text_hash) · register_signer(eth_address) · revoke_signer · freeze() · unfreeze()   owner only
propose(action, args_json) -> proposal_id                                                       agent key
execute(proposal_id, req_bytes, resp_bytes, sig_65) -> Promise
   checks: signer allowlisted · sha256(req):sha256(resp) recovers signer · verdict == ALLOW ·
           action == canonical(proposal) · nonce unused · not expired · within caps · not frozen

// Firewall core library (Furqaan) — the seam; William's API imports it
judge(mandate_text, action, args_json, context) ->
      { verdict, reason, model, chat_id, req_bytes, resp_bytes, signature, signer }
proposeOnChain(action, args_json) -> { proposal_id, near_tx_hash }
executeOnChain(proposal_id, req_bytes, resp_bytes, signature) -> { near_tx_hash, ok, error? }

// 1Click module (William)
payCrossChain({ amount, origin_asset, dest_asset, recipient, refund_to })
      -> { deposit_address, quote }      // contract executes ft_transfer to deposit_address
trackDeposit(deposit_address) -> { status, dest_tx_hash? }

// HTTP API (William builds; UI + agent use it)
POST /api/mandate          { text }                          -> { mandate_id, text_hash }
POST /api/propose          { mandate_id, action, args, context } -> { proposal_id }
GET  /api/proposals        ?since=ts                         -> [ Proposal ]
GET  /api/proposals/:id                                      -> Proposal
GET  /api/redteam/latest                                     -> { attacks_blocked, attacks_total,
                                                                  benign_denied, benign_total, cases[] }
Proposal = { id, created_at, action, args, context_excerpt,
             status: "pending"|"allowed"|"denied"|"executed"|"failed",
             verdict, reason, model, chat_id, signature, signer, attestation_url,
             near_tx_hash?, oneclick: { deposit_address, status, dest_tx_hash? }? }
```

6. Day 1 (Nov 1): kill tests

Both top ideas depend on one untested step: verifying a real NEAR AI signature inside a NEAR contract. Day 1 proves that before anything else gets built. Meanwhile William proves the 1Click payment path and scaffolds the API and UI.

Test	Owner / hours	Exact steps	Pass	If it fails
KT1 Signed verdict	F 0–3h	Call `GET /v1/models` and confirm Qwen3.8-27B is ready. Then `POST /v1/chat/completions` (non-streaming, JSON output), saving the **raw** request and response bytes. Then `GET /v1/signature/{id}?model=…&signing_algo=ecdsa`, and `GET /v1/attestation/report` with a fresh nonce. Locally, run `ethers.verifyMessage(sha256(req)+":"+sha256(resp), sig)`.	The recovered address equals the signing address, and the report echoes our nonce.	Try `ed25519`, and check whether the JSON is canonicalized. At 6h, ask NEAR AI in Discord and plan a relayer fallback.
KT2 On-chain verify	F 3–8h	Write a minimal testnet contract `verify(req, resp, sig)` that hashes both, builds the EIP-191 message, runs `keccak256` and `ecrecover`, and compares against an allowlist. Run it with the KT1 artifacts. Also add a function-call key and try calling another contract with it.	`true` for the real pair and `false` with one byte flipped. Gas fits in one call, including with a 4 KB response. The out-of-scope call is rejected.	Cap the response length, or use `env::ed25519_verify`. If both fail, use a relayer with its own key and state that honestly.
KT3 Model quality	W writes cases + a quick runner script, F supplies the prompt, 8–11h	20 malicious cases (swapped recipient, 10× amount, unknown token, approval to an unknown contract, poisoned 402 paywall) and 20 benign ones. Run them on Qwen3.8-27B, then on GLM-5.3-flash.	≥18/20 attacks denied, ≤2/20 benign denied, median latency <5 s.	Tighten the schema and mandate format and retest on Day 2. If it still fails, **switch to Sealed-lite**.
KT4 Settlement	W 2–6h	Using the 1Click TypeScript SDK or curl: `GET /v0/tokens`, then `POST /v0/quote` with `dry:true` for about $2 NEAR USDC → Base. Then a real quote. Furqaan sends the `ft_transfer` to the deposit address. Then `/v0/deposit/submit` and polling `/v0/status`.	Funds arrive at William's Base address in about a minute. Status reaches its success state.	Raise the amount or change the route; minimums vary. If quotes fail entirely, escalate in Discord.

Decision rule (both, end of Nov 1): if KT1, KT2 and KT3 pass, commit to Intented and stop evaluating other ideas. If KT2 fails, keep Intented with the relayer fallback. If KT3 fails twice, switch to **Sealed-lite**: a commit-reveal prediction-market escrow, with 1Click funding and the same signed-verdict verifier as the resolver. The UI work carries over (feed + receipt views), so no Day-1 work is wasted.

7. Day-by-day plan (Nov 1 – Nov 14)

Day	FURQAAN	WILLIAM	Sync / checkpoint
1 Nov 1	Create the repo. Run KT1 and KT2, and supply the KT3 prompt.	Run KT4 (1Click). Write the KT3 fixtures and runner script. Scaffold `/api` and `/web`.	Morning: lock the interface spec. Evening: go/no-go.
2 Nov 2	Contract skeleton: `propose`, `execute`, signer registry, nonce and expiry guard, caps, agent key.	API server: all endpoints, SQLite schema, and a firewall-core stub returning fake signed verdicts.	Retest KT3 if needed.
3 Nov 3	Verdict binding in the contract: recover the signer, check the action matches, parse the verdict.	1Click module: quote → deposit address → submit → status polling → refund, with status written to proposals.	—
4 Nov 4	Unit and near-workspaces sandbox tests. Attestation checker that registers the signer.	Mock x402 paywall (honest + poisoned). UI v1: proposal feed against the API.	First allowed action executes on testnet (from a script).
5 Nov 5	Firewall core: canonical encoder, prompt layout, NEAR AI call, signature fetch, `proposeOnChain` / `executeOnChain`.	Demo agent: calls the paywall, reads the 402, builds the action, calls `/api/propose`. Receipt view in the UI.	Cross-test: core-encoded bytes = contract re-derived bytes.
6 Nov 6	Contract payout path to the 1Click deposit address. **Swap the stub for the real firewall core** in William's API.	Mandate editor. Landing section and branding. 1Click status in the receipt.	The API now produces real signed verdicts.
7 Nov 7	BOTH **Integration day.** Full flow on testnet: agent → API → firewall core → contract verifies → execute → 1Click → the UI shows the receipt. Mainnet 1Click payment with a small amount. Fix any interface mismatches.		**Checkpoint: the full flow works in the UI.** If not, cut scope (section 9).
8 Nov 8	Prompt tuning against the red-team numbers. Start hardening: injection-resistant layout.	`make redteam` runner over the real pipeline, plus the `/api/redteam/latest` endpoint and the UI stats panel.	Review the red-team numbers together.
9 Nov 9	Hardening: signer rotation, edge cases (expired, replayed, oversized), freeze switch.	Deploy the API (Railway/Render) and the UI (Vercel) with the domain. Add 10+ attack cases that target the firewall itself. First demo rehearsal.	—
10 Nov 10	Security self-review. Optional OutLayer integration only if Discord Q3 says yes.	API: error handling, retries, rate limits, logs. UI polish: empty, loading and error states, mobile, OG image.	Feature-complete on testnet.
11 Nov 11	**Mainnet contract deploy** (confirm together first). Register the signer on mainnet. Fund wallets.	Point the API at mainnet. Run the allowed and blocked demo through the UI. Capture tx hashes and screen recordings.	Two mainnet tx hashes: one allowed, one blocked.
12 Nov 12	README: architecture diagram, threat model, trust assumptions, contract and core docs.	README: problem, product, "do X - see Y", API reference, red-team method, screenshots and GIFs.	Each reviews the other's half.
13 Nov 13	**Feature freeze.** Fresh-clone test following the README exactly. Secret scan.	Record and edit the demo video. Draft the X post. Fix any API or UI bugs from the fresh-clone test.	Video approved by both.
14 Nov 14	Final repo check: public, license, no secrets. **Submit by about 20:00 IST** (team size 2, tracks security, agents, privacy, payout account).	Publish the X post. Check that the UI, API and video links work logged-out.	Go/no-go call before submitting. **Submitting is irreversible.**

8. Discord questions to settle before Nov 1

#	Question	Channel	Asks
1	Do proprietary models (Claude, GPT) on NEAR AI Cloud return TEE signatures, or only NEAR-hosted open models? How long are `/v1/signature/{id}` results kept, and how often do signing addresses rotate?	NEAR AI	F
2	Are there hackathon credits for NEAR AI Cloud? Can it be paid in crypto?	NEAR AI	F
3	Is there an API for OutLayer multisig approvals (so a bot can sign), or a co-signer or webhook hook? Can `request_execution` make outbound HTTPS calls?	OutLayer	F
4	After the Oct 7 `Notify` removal, does `ft_withdraw` or `mt_withdraw` with `msg` still trigger `ft_on_transfer`? Can 1Click deliver to a contract with a custom msg?	NEAR Intents	F
5	How long does partner JWT approval take? Is there a hackathon fee waiver? Is the Sep 9 $1,000 temporary minimum still active, and on which routes?	NEAR Intents	W
6	What's the difference between `confidentiality: "basic"` and `"advanced"`? Can a contract be a confidential recipient? Is there a view-key API? (Matters for the Sealed-lite fallback.)	NEAR Intents	F
8	Is there an official successor to Shade Agents that judges expect?	NEAR dev	F
9	Can one project win prizes in more than one track? Can the grand-prize winner keep a track prize?	Hackathon	W
10	Are throwaway experiments and design mockups from before Nov 1 fine if the repo starts on Nov 1? Do the AI judges run the code? Does a mainnet deploy count for more?	Hackathon	W
11	Is KYC or a sanctions check needed for NEAR payouts (India-based winners)? What NEAR/USD rate is used?	Hackathon	W

Question 7 from the research report (Rhea lending status) only mattered for the rejected Never-Liq idea, so it's dropped.

9. Risks, cut lines & fallbacks

Risk	Mitigation / cut line	Owner
NEAR AI signature won't verify on-chain (KT2)	Relayer with its own allowlisted key verifies off-chain. State the trust assumption honestly in the README.	F
Model misjudges attacks or normal requests (KT3)	Tighter canonical schema, a mandate template, and quoted untrusted data. Try the second model. If it fails twice, switch to Sealed-lite.	F + W
Firewall model gets prompt-injected	Structured output, untrusted input quoted as data, contract caps as the hard backstop. William's fixtures and runner include attacks that target the firewall itself.	F + W
1Click route minimums or failures	Raise the amount or switch routes. If cross-chain is blocked, demo the NEAR-native `ft_transfer` and show a 1Click quote only.	W
Firewall core or contract slips, so the API has nothing real to call	The API and UI already run on the stub core. The swap is fixed at Day 6, with integration on Day 7. If the core isn't ready, record the demo with real testnet transactions from scripts and use the UI as the viewer.	F + W
Signing key rotates mid-demo	Fetch signatures immediately. Keep `register_signer` callable. Re-run the attestation checker before recording.	F
Integration slips past Day 7	Cut in this order: optional OutLayer integration → mandate editor UI → daily cap → the cross-chain leg (keep the NEAR-native payment). Never cut the attack demo.	BOTH
Similar entries (rule-based firewalls such as AgentSentinel and Guardia)	Lead with *semantic* checks plus on-chain verified signatures, and published red-team numbers. The demo shows a case that rule-based tools pass but Intented blocks.	W

Risk	Mitigation / cut line	Owner
Submission mistakes (irreversible)	A two-person checklist (section 10). Submit about 9 hours before the deadline.	BOTH

10. Submission checklist (Nov 14)

Item	Owner
☐ Repo public, open-source license, no secrets or .env committed (run a secret scan)	F
☐ README technical: architecture, threat model, trust assumptions, contract and core docs, mainnet tx hashes	F
☐ README product: what it is, problem, "do X - see Y", track hints named, API reference, red-team metrics, screenshots and GIFs	W
☐ One-command test run works from a fresh clone	F
☐ API and UI deployed, and the links work without logging in	W
☐ Demo video (about 2:00–2:20) uploaded and public	W
☐ X post live: "built at NEAR HACKATHON", video attached, repo linked	W
☐ Description (20–2,000 characters) written for machine readers, no marketing	F
☐ Form fields: team size 2; tracks security, agents, privacy; NEAR mainnet payout account; repo URL; X URL	F
☐ Final go/no-go call, then submit **once** by about 20:00 IST Nov 14	BOTH

11. Working agreement

- **Daily sync (15 min):** what shipped, what's blocked, whether the interface spec changed. Post a one-line async update if a call isn't possible.
- **Repo layout:** `/contract` and `/firewall-core` (Furqaan); `/api`, `/web`, `/agent` and `/redteam` (William). Each of you merges into your own folders. Changes to the interface spec need the other's review.
- **Branches:** `main` stays demo-able. Each feature gets a branch.
- **Secrets:** the NEAR AI key, JWT and mainnet keys stay in a shared password manager, never in the repo. William's frontend never holds private keys.
- **Mainnet actions** (deploys, funded transactions) need a quick confirmation from both of you first.
- **Decisions log** in `DECISIONS.md`: interface changes, model choice, cut-line calls.

Sources: nearhackathon.com and HACKATHON.md; NEAR AI Cloud and 1Click docs; live API checks on Oct 22 2026. Full evidence is in "NEAR hackathon idea selection" (Downloads/reports). Domain availability was checked Oct 22 and can change.